

Name: Monisha Raghini Chelle

RollNo:2520030537

Section -6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

19.To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14  Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                          OSSP-Practical  example.txt            hello       moni     sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello      hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c    hello:Zone.Identifier    hello_shortcut  skill        virtual_address
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the Signal Handlers Program

Run:

nano signal_handlers.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano signal_handlers.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Paste this complete C code:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 signal_handlers.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
#include <errno.h>

static volatile sig_atomic_t sigint_count = 0;
static volatile sig_atomic_t sigusr1_received = 0;

/* Signal handlers only update flags */
void handle_sigint(int signo)
{
    (void)signo;
    sigint_count++;
}

void handle_sigusr1(int signo)
{
    (void)signo;
    sigusr1_received = 1;
}

int main(void)
{
    struct sigaction sa_int;
    struct sigaction sa_usr1;

    memset(&sa_int, 0, sizeof(sa_int));
    memset(&sa_usr1, 0, sizeof(sa_usr1));

    /* Register the SIGINT handler */
    sa_int.sa_handler = handle_sigint;
    sigemptyset(&sa_int.sa_mask);
    sa_int.sa_flags = 0;

    if (sigaction(SIGINT, &sa_int, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }

    /* Register the SIGUSR1 handler */
    sa_usr1.sa_handler = handle_sigusr1;
    sigemptyset(&sa_usr1.sa_mask);
    sa_usr1.sa_flags = 0;

    if (sigaction(SIGUSR1, &sa_usr1, NULL) == -1)
    {
        perror("sigaction SIGUSR1");
        return 1;
    }

    printf("Q19 - Signal Handlers Program\n");
    printf("PID: %ld\n", (long)getpid());
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 signal_handlers.c *
{
    perror("sigaction SIGINT");
    return 1;
}

/* Register the SIGUSR1 handler */
sa_usr1.sa_handler = handle_sigusr1;
sigemptyset(&sa_usr1.sa_mask);
sa_usr1.sa_flags = 0;

if (sigaction(SIGUSR1, &sa_usr1, NULL) == -1)
{
    perror("sigaction SIGUSR1");
    return 1;
}

printf("Q19 - Signal Handlers Program\n");
printf("PID: %ld\n", (long)getpid());
printf("SIGINT and SIGUSR1 handlers registered.\n");

/* Generate a test signal */
printf("\nSending SIGUSR1 to this process...\n");

if (kill(getpid(), SIGUSR1) == -1)
{
    perror("kill");
    return 1;
}

if (sigusr1_received)
{
    printf("SIGUSR1 handled successfully.\n");
    sigusr1_received = 0;
}

printf("\nPress Ctrl+C twice to test SIGINT.\n");
printf("The program will remain active after the first Ctrl+C.\n");
fflush(stdout);

while (sigint_count < 2)
{
    pause();

    if (sigint_count == 1)
    {
        printf("\nSIGINT received. Program is still running.\n");
        printf("Press Ctrl+C again to exit.\n");
        fflush(stdout);
    }
}

printf("\nSIGINT received twice.\n");
printf("Signal handling test completed successfully.\n");

return 0;
}
```

Save using Ctrl + O, Enter, then Ctrl + X.

```
Write to File: signal_handlers.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^G Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Compile:

gcc -Wall -Wextra signal_handlers.c -o signal_handlers

Verify:

ls -l signal_handlers.c signal_handlers

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano signal_handlers.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra signal_handlers.c -o signal_handlers
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l signal_handlers.c signal_handlers
-rwxr-xr-x 1 monis monis 16600 Sep 26 22:12 signal_handlers
-rw-r--r-- 1 monis monis 2093 Sep 26 22:11 signal_handlers.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 3 — Test Signal Handlers

Run:

`./signal_handlers`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./signal_handlers
Q19 - Signal Handlers Program
PID: 1181
SIGINT and SIGUSR1 handlers registered.

Sending SIGUSR1 to this process...
SIGUSR1 handled successfully.

Press Ctrl+C twice to test SIGINT.
The program will remain active after the first Ctrl+C.
```

Press **Ctrl + C** once.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./signal_handlers
Q19 - Signal Handlers Program
PID: 1181
SIGINT and SIGUSR1 handlers registered.

Sending SIGUSR1 to this process...
SIGUSR1 handled successfully.

Press Ctrl+C twice to test SIGINT.
The program will remain active after the first Ctrl+C.
^] ^] ^] ^C
SIGINT received. Program is still running.
Press Ctrl+C again to exit.
```

Press Ctrl + C a second time.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill$ ./signal_handlers
Q19 - Signal Handlers Program
PID: 1181
SIGINT and SIGUSR1 handlers registered.

Sending SIGUSR1 to this process...
SIGUSR1 handled successfully.

Press Ctrl+C twice to test SIGINT.
The program will remain active after the first Ctrl+C.
^] ^] ^] ^C
SIGINT received. Program is still running.
Press Ctrl+C again to exit.
^] ^] ^] ^] ^] ^] ^] ^] ^] ^] ^C
SIGINT received twice.
Signal handling test completed successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Create sigint_forwarding.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano sigint_forwarding.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 sigint_forwarding.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>

static volatile sig_atomic_t interrupt_received = 0;
static volatile sig_atomic_t foreground_pgid = 0;

typedef enum {
    JOB_EMPTY,
    JOB_RUNNING,
    JOB_COMPLETED,
    JOB_INTERRUPTED
} JobState;

typedef struct {
    int id;
    pid_t pid;
    pid_t pgid;
    JobState state;
} Job;

static Job job = {0};
static int next_job_id = 1;

/* Capture Ctrl+C without terminating the parent shell. */
static void handle_sigint(int signo)
{
    (void)signo;

    interrupt_received = 1;

    /*
     * Forward SIGINT to the foreground job's process group.
     * kill() is async-signal-safe.
     */
    if (foreground_pgid > 0)
        kill(-(pid_t)foreground_pgid, SIGINT);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 sigint_forwarding.c *

static const char *state_name(JobState state)
{
    switch (state) {
        case JOB_RUNNING:
            return "RUNNING";

        case JOB_COMPLETED:
            return "COMPLETED";

        case JOB_INTERRUPTED:
            return "INTERRUPTED";

        default:
            return "EMPTY";
    }
}

static void show_job(void)
{
    if (job.state == JOB_EMPTY) {
        printf("No job record found.\n");
        return;
    }

    printf("\nJOB TABLE\n");
    printf("ID\tPID\tPGID\tSTATE\n");

    printf("%d\t%d\t%d\t%s\n",
        job.id,
        (long)job.pid,
        (long)job.pgid,
        state_name(job.state));
}

static void start_foreground_job(int seconds)
{
    if (seconds < 0) {
        printf("Invalid duration.\n");
        return;
    }

    if (job.state == JOB_RUNNING) {
        printf("A foreground job is already running.\n");
        return;
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 sigint_forwarding.c *
    return;
}

/*
 * Block SIGINT while creating and registering the child.
 * This prevents a race before foreground_pgid is assigned.
 */
sigset_t block_set;
sigset_t old_set;

sigemptyset(&block_set);
sigaddset(&block_set, SIGINT);

if (sigprocmask(SIG_BLOCK, &block_set, &old_set) == -1) {
    perror("sigprocmask");
    return;
}

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    sigprocmask(SIG_SETMASK, &old_set, NULL);
    return;
}

if (pid == 0) {
    /* Child belongs to its own process group. */
    if (setpgid(0, 0) == -1) {
        perror("setpgid");
        _exit(1);
    }

    /* Restore normal Ctrl+C behavior in the child. */
    struct sigaction default_action;

    memset(&default_action, 0, sizeof(default_action));
    default_action.sa_handler = SIG_DFL;
    sigemptyset(&default_action.sa_mask);

    sigaction(SIGINT, &default_action, NULL);

    /* Restore the original signal mask. */
    sigprocmask(SIG_SETMASK, &old_set, NULL);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

```
    char duration[20];

    snprintf(duration, sizeof(duration),
              "%d", seconds);

    execlp("sleep", "sleep",
           duration, (char *)NULL);

    perror("execlp");
    _exit(127);
}

/*
 * The parent also attempts setpgid to avoid
 * a process-group creation race.
 */
if (setpgid(pid, pid) == -1 && errno != EACCES)
    perror("setpgid");

job.id = next_job_id++;
job.pid = pid;
job.pgid = pid;
job.state = JOB_RUNNING;

foreground_pgid = (sig_atomic_t)pid;
interrupt_received = 0;

if (sigprocmask(SIG_SETMASK, &old_set, NULL) == -1)
    perror("sigprocmask");

printf("\nForeground job [%d] started.\n", job.id);
printf("PID: %ld\n", (long)job.pid);
printf("PGID: %ld\n", (long)job.pgid);
printf("Running sleep %d...\n", seconds);
printf("Press Ctrl+C to interrupt the foreground job.\n");
fflush(stdout);

int status;
pid_t result;

do {
    result = waitpid(pid, &status, 0);
} while (result == -1 && errno == EINTR);
```

```
/*
 * Block SIGINT again while clearing the active job
 * to prevent forwarding a signal to an old PGID.
 */
if (sigprocmask(SIG_BLOCK, &block_set, &old_set) == -1)
    perror("sigprocmask");

foreground_pgid = 0;

if (result == -1) {
    perror("waitpid");
    job.state = JOB_INTERRUPTED;
}
else if (WIFSIGNALED(status)) {
    job.state = JOB_INTERRUPTED;

    printf("\nForeground job terminated by signal %d.\n",
        WTERMSIG(status));
}
else if (WIFEXITED(status)) {
    job.state = JOB_COMPLETED;

    printf("\nForeground job completed.\n");
    printf("Exit status: %d\n",
        WEXITSTATUS(status));
}

if (sigprocmask(SIG_SETMASK, &old_set, NULL) == -1)
    perror("sigprocmask");

if (interrupt_received)
    printf("SIGINT was captured by the shell.\n");

printf("Shell process is still running.\n");

show_job();
}

int main(void)
{
    struct sigaction sa;
    char input[100];
    int seconds;
    char extra;
```

```
memset(&sa, 0, sizeof(sa));

sa.sa_handler = handle_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

if (sigaction(SIGINT, &sa, NULL) == -1) {
    perror("sigaction");
    return 1;
}

printf("Q19 - SIGINT Forwarding Program\n");
printf("Shell PID: %ld\n", (long)getpid());
printf("\nCommands:\n");
printf("run NUMBER\n");
printf("jobs\n");
printf("exit\n");

while (1) {
    printf("\nshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL) {
        if (ferror(stdin) && errno == EINTR) {
            clearerr(stdin);
            continue;
        }
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strcmp(input, "jobs") == 0) {
        show_job();
    }
    else if (sscanf(input, "run %d %c",
                    &seconds, &extra) == 1 &&
              seconds >= 0) {
        start_foreground_job(seconds);
    }
    else {
```

^G Help
^X Exit

^O Write Out
^R Read File

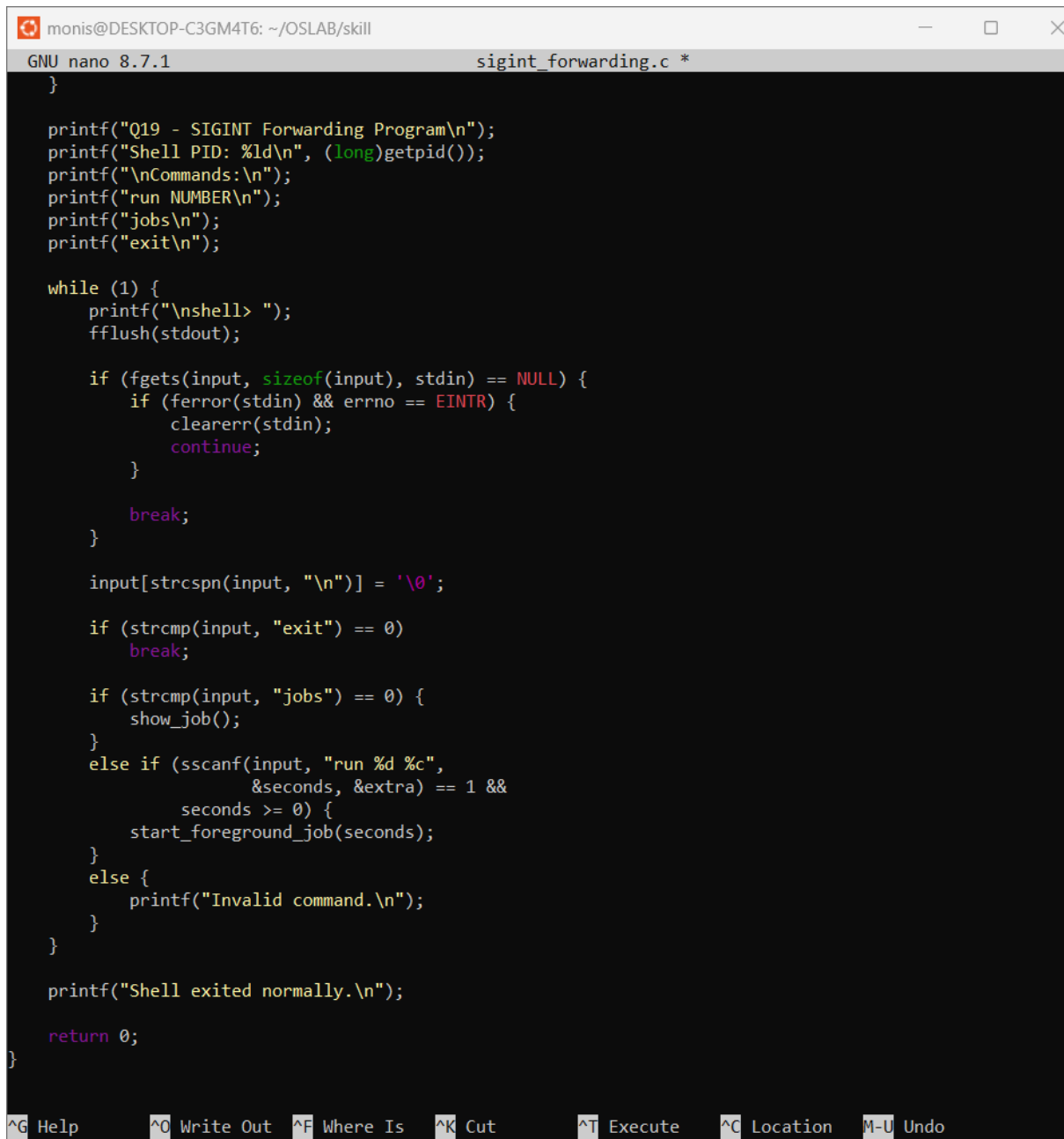
^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo



```
GNU nano 8.7.1 sigint_forwarding.c *
}

printf("Q19 - SIGINT Forwarding Program\n");
printf("Shell PID: %ld\n", (long)getpid());
printf("\nCommands:\n");
printf("run NUMBER\n");
printf("jobs\n");
printf("exit\n");

while (1) {
    printf("\nshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL) {
        if (ferror(stdin) && errno == EINTR) {
            clearerr(stdin);
            continue;
        }
        break;
    }

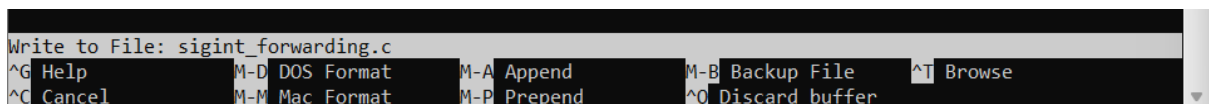
    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strcmp(input, "jobs") == 0) {
        show_job();
    }
    else if (sscanf(input, "run %d %c",
                    &seconds, &extra) == 1 &&
             seconds >= 0) {
        start_foreground_job(seconds);
    }
    else {
        printf("Invalid command.\n");
    }
}

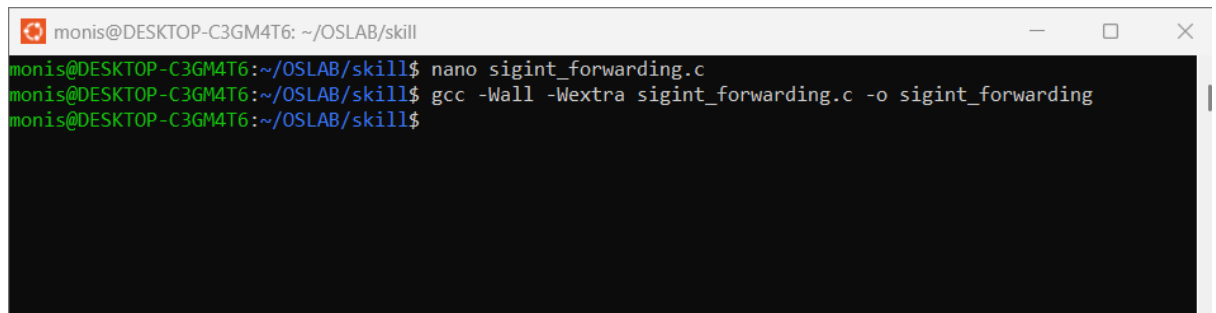
printf("Shell exited normally.\n");
return 0;
}
```

Save the file using Ctrl + O, Enter, then Ctrl + X.



Compile it:

gcc -Wall -Wextra sigint_forwarding.c -o sigint_forwarding

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. It shows three commands being executed: 'nano sigint_forwarding.c', 'gcc -Wall -Wextra sigint_forwarding.c -o sigint_forwarding', and the prompt 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'.

Step 5 — Test SIGINT Forwarding

Run:

`./sigint_forwarding`

Enter:

`run 60`

While the foreground job is running, press Ctrl + C.

Next, enter:

`jobs`

Now test normal completion:

`run 3`

Finally, enter:

`exit`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./sigint_forwarding
Q19 - SIGINT Forwarding Program
Shell PID: 1428

Commands:
run NUMBER
jobs
exit

shell> run 60

Foreground job [1] started.
PID: 1429
PGID: 1429
Running sleep 60...
Press Ctrl+C to interrupt the foreground job.
^] ^C
Foreground job terminated by signal 2.
SIGINT was captured by the shell.
Shell process is still running.

JOB TABLE
ID      PID      PGID      STATE
1       1429     1429     INTERRUPTED

shell> jpbs
Invalid command.

shell> run 3

Foreground job [2] started.
PID: 1432
PGID: 1432
Running sleep 3...
Press Ctrl+C to interrupt the foreground job.

Foreground job completed.
Exit status: 0
Shell process is still running.

JOB TABLE
ID      PID      PGID      STATE
2       1432     1432     COMPLETED

shell> exit
Shell exited normally.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

ls -l signal_handlers.c signal_handlers \ sigint_forwarding.c sigint_forwarding

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l signal_handlers.c signal_handlers \
    sigint_forwarding.c sigint_forwarding
-rwxr-xr-x 1 monis monis 17408 Sep 26 22:22 sigint_forwarding
-rw-r--r-- 1 monis monis  6175 Sep 26 22:22 sigint_forwarding.c
-rwxr-xr-x 1 monis monis 16600 Sep 26 22:12 signal_handlers
-rw-r--r-- 1 monis monis  2093 Sep 26 22:11 signal_handlers.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```